

Artificial Intelligence

Prof. Chang-Hee Han



Introduction to Keras and TensorFlow

3.0 This chapter covers

- A closer look at TensorFlow, Keras, and their relationship
- Setting up a deep learning workspace
- An overview of how core deep learning concepts translate to Keras and TensorFlow

3.1 What's TensorFlow?

▣ Introduction to TensorFlow

- Free, open-source machine learning platform
- Based on Python, developed by Google
- Like NumPy, works with tensors (mathematical data)
- More powerful features:
  - ➢ Automatic gradient calculation for ML
  - ➢ Runs on CPUs, GPUs, and TPUs
  - ➢ Supports distributed computation
  - ➢ Can export to C++, JavaScript, TensorFlow Lite

3.1 What's TensorFlow?

TensorFlow Ecosystem & Scalability

- TensorFlow as a platform with many components:
  - TF-Agents → reinforcement learning
  - TFX → workflow management
  - TensorFlow Serving → production deployment
  - TensorFlow Hub → pre-trained models
- Scales to very large models:
  - 1.1 exaFLOPS weather forecasting on 27,000 GPUs (IBM Summit)
  - Google used TensorFlow for AlphaZero and other deep learning apps
- With TPUs or GPU clusters:
  - Can reach ~10 petaFLOPS
  - Still only ~1% of the top supercomputer (2019)

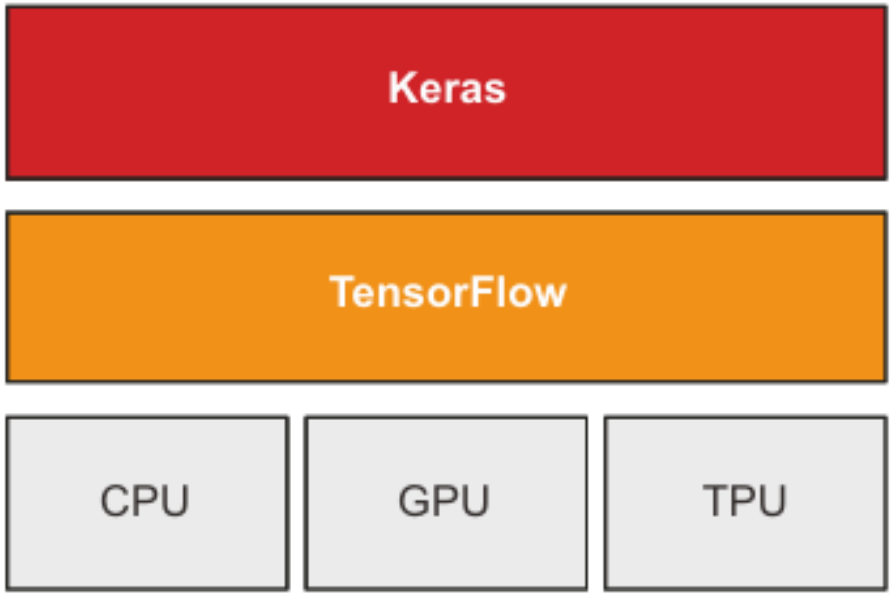
3.2 What's Keras?

▣ What is Keras?

- Deep learning API for Python, built on TensorFlow
- Designed for fast and easy experimentation
- Runs on CPU, GPU, TPU, scalable to thousands of machines
- Human-centered API → simple, consistent, helpful feedback
- Easy for beginners, productive for experts
- Widely used: over 1M users (Google, Netflix, Uber, NASA, Kaggle, etc.)
- Examples: YouTube recommendations, Waymo self-driving cars

3.2 What's Keras?

▣ What is Keras?



Deep learning development: layers, models, optimizers, losses, metrics...

Tensor manipulation infrastructure: tensors, variables, automatic differentiation, distribution...

Hardware: execution

Figure 3.1 Keras and TensorFlow: TensorFlow is a low-level tensor computing platform, and Keras is a high-level deep learning API

3.2 What's Keras?

▣ Why Keras is Powerful

- Supports different workflows:
  - ➢ High-level (like Scikit-learn, just call fit())
  - ➢ Low-level (like NumPy, full control)
- Flexible for students → researchers → engineers
- No need to switch frameworks as you grow
- Similar to Python philosophy:
  - ➢ Multiple ways to solve problems
  - ➢ Works well across many industries and tasks
- Keras = the Python of Deep Learning

3.3 Keras and TensorFlow: A brief history

▣ Keras & TensorFlow Timeline

- Keras released first (March 2015) → built on Theano
- TensorFlow released (Nov 2015) → later became Keras backend
- 2015–2017: Keras supported multiple backends (Theano, TensorFlow, CNTK, MXNet)
- Eventually, TensorFlow became the main backend
- Symbiotic growth:
- Keras brought many users into TensorFlow
- By 2017, most TensorFlow users worked with Keras
- 2018: Keras chosen as TensorFlow's official high-level API
- 2019: TensorFlow 2.0 → redesigned with Keras API at its core

3.4 Setting up a deep learning workspace

▣ Setting Up a Deep Learning Environment

- There are three main ways to use a GPU for deep learning:
  - ➢ Purchase an NVIDIA GPU and install it on your computer.
  - ➢ Use a GPU instance from cloud services like Google Cloud or AWS EC2.
  - ➢ Use the free GPU runtime provided by Google's notebook service, Colab (Colaboratory).

3.4 Setting up a deep learning workspace

▣ Setting Up a Deep Learning Environment

- Whether running locally or in the cloud, it is best to use a Unix workstation.
- Running Keras on Windows is technically possible, but not recommended.
- If you want to use Windows for deep learning:
  - ➢ Set up a dual boot with Ubuntu, or
  - ➢ Use WSL (Windows Subsystem for Linux) to handle everything easily.
- WSL is a compatibility layer that allows running Linux applications on Windows.
- Though it may seem complex at first, this setup saves time and prevents issues in the long run.

3.4 Setting up a deep learning workspace

▣ Jupyter Notebook: Recommended Tool for Deep Learning

- Jupyter Notebook is a great way to run deep learning experiments.
- Widely used in the data science and machine learning community.
- Notebooks can be created via the Jupyter app (https://jupyter.org) and edited in the browser.
- Supports running Python code and adding rich text notes.
- Experiments can be broken into small, independent cells.
- Interactive: if an error occurs, you only need to re-run the affected cells, not the entire code.
- While independent Python scripts or IDEs (e.g., PyCharm) can be used, Jupyter Notebook is recommended for starting with Keras.
- All code examples from this book are available as open-source notebooks:
  👉 https://github.com/rickiepark/deep-learning-with-python-2nd

3.4 Setting up a deep learning workspace

▣ Using Colab

- Colab (Colaboratory) is a free Jupyter Notebook service that runs fully in the cloud, with no installation required.
- You can write and run Keras scripts directly in a web page.
- Provides free (but limited) GPU and TPU runtimes, so you don’t need to buy your own GPU.
- Colab is the recommended way to run the code examples in this book.

3.4 Setting up a deep learning workspace

▣ Using Colab

- To use Colab, visit https://colab.research.google.com and click the New Notebook button.
- You will see a notebook interface similar to Figure 3-2.

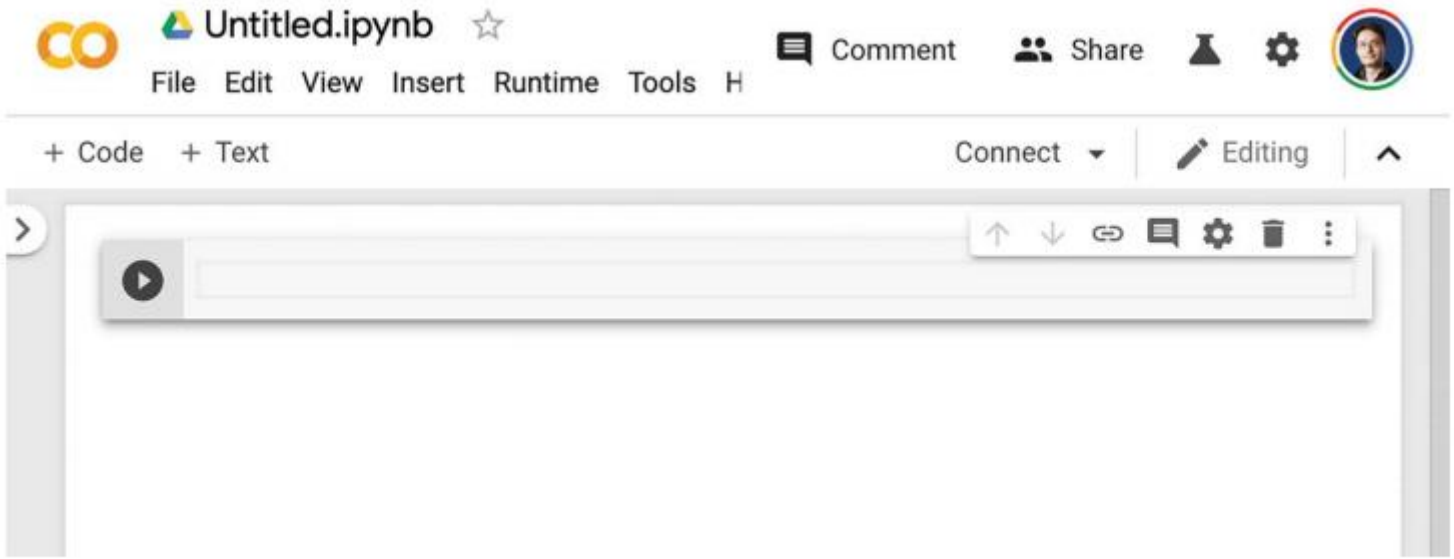


Figure 3.2 A Colab notebook

3.4 Setting up a deep learning workspace

▣ Using Colab

- The toolbar has two buttons: + Code and + Text.
- Code cells are for runnable Python code; Text cells are for notes or explanations.
- To run code in a cell, type your code and press Shift + Enter (see Figure 3-3).

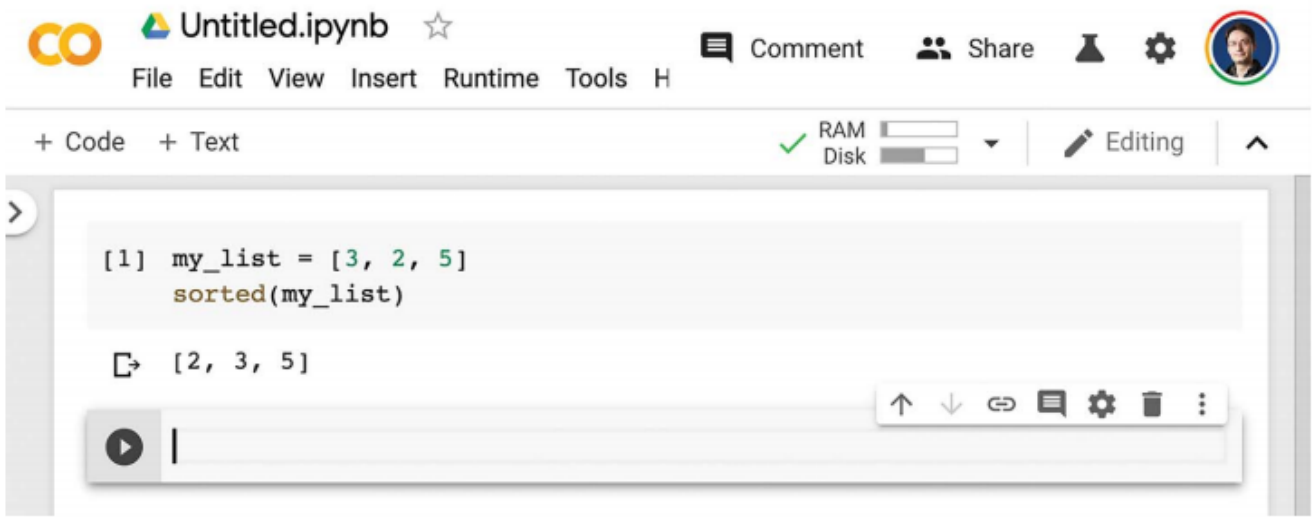


Figure 3.3 Creating a code cell

3.4 Setting up a deep learning workspace

▣ Using Colab

- In text cells, you can use Markdown syntax (see Figure 3-4).
- Press Shift + Enter in a text cell to render the content on screen.

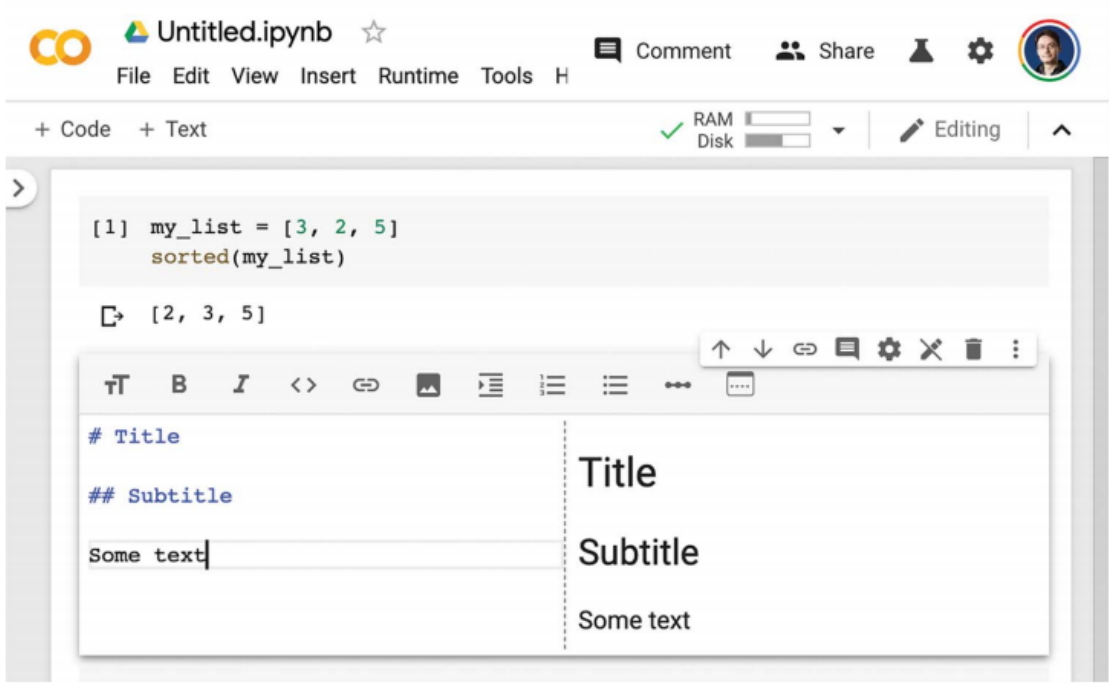


Figure 3.4 Creating a text cell

3.4 Setting up a deep learning workspace

Using Colab

- In text cells, you can add structured text to your notebook.
- You can write section titles, long explanations, or even insert images.
- Jupyter notebooks support multimedia content.

Installing Packages with pip

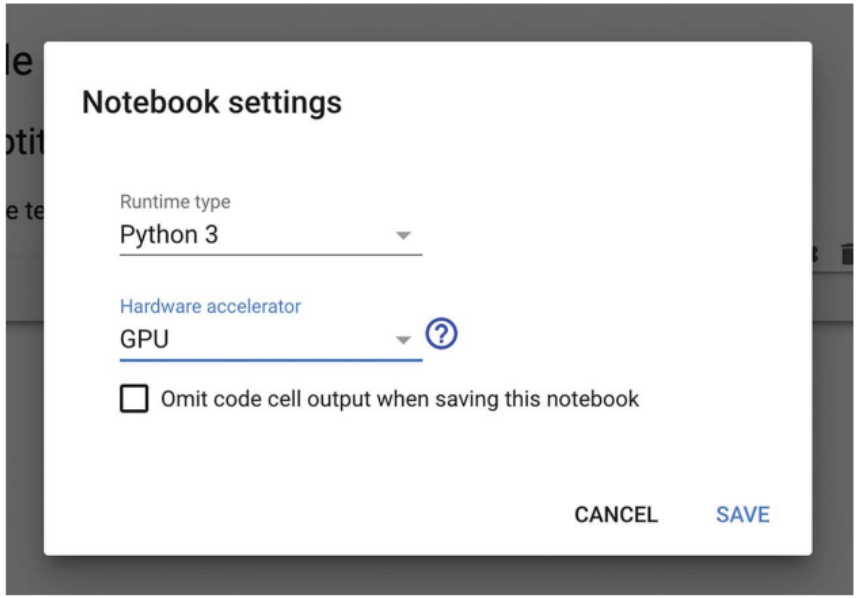
- In the default Colab environment, TensorFlow and Keras are already installed.
- You can start right away without extra installation.
- If you need more packages, use pip commands in code cells:

```
!pip install package_name
```

3.4 Setting up a deep learning workspace

▣ Using Colab - GPU Runtime

- To use a GPU runtime in Colab:
  - ➢ Go to the menu → Runtime → Change runtime type.
  - ➢ Set the hardware accelerator to GPU (see Figure 3-5).



3.4 Setting up a deep learning workspace

▣ Using Colab

- TensorFlow and Keras will automatically run on GPU if one is available.
- After selecting a GPU runtime, no extra setup is required.
- Under Hardware accelerator, you can also choose TPU runtime.
- Unlike GPU runtime, when using TPU in TensorFlow/Keras, you need to make small code changes.
- For now, use the GPU runtime to run the book's code examples.

3.5 First steps with TensorFlow

▣ Getting Started with TensorFlow

- Neural network training is based on the following key concepts:

① Tensors and Tensor Operations

- Tensor: a special object that stores the state of the network (includes variables).
- Operations: functions like **addition, ReLU, matmul**.

② Gradient Calculation (Backpropagation)

- Gradients of mathematical expressions are computed using **GradientTape** in TensorFlow.

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- To do any operation in TensorFlow, you need a tensor.
- Creating a tensor requires an initial value.
  - ➢ Examples:
    - Create a tensor full of ones or zeros (Code 3-1).

Listing 3.1 All-ones or all-zeros tensors

```
>>> import tensorflow as tf
>>> x = tf.ones(shape=(2, 1))
>>> print(x)
tf.Tensor(
[[1.]
 [1.]], shape=(2, 1), dtype=float32)
>>> x = tf.zeros(shape=(2, 1))
>>> print(x)
tf.Tensor(
[[0.]
 [0.]], shape=(2, 1), dtype=float32)
```

Equivalent to np.ones(shape=(2, 1))

Equivalent to np.zeros(shape=(2, 1))

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- Examples:
  - Create a tensor with values drawn from a random distribution (Code 3-2).

Listing 3.2 Random tensors

```
>>> x = tf.random.normal(shape=(3, 1), mean=0., stddev=1.)
>>> print(x)
tf.Tensor(
[[-0.14208166]
 [-0.95319825]
 [ 1.1096532 ]], shape=(3, 1), dtype=float32)
>>> x = tf.random.uniform(shape=(3, 1), minval=0., maxval=1.)
>>> print(x)
tf.Tensor(

[[0.33779848]
 [0.06692922]
 [0.7749394 ]], shape=(3, 1), dtype=float32)
```

Tensor of random values drawn from a normal distribution with mean 0 and standard deviation 1. Equivalent to np.random.normal(size=(3, 1), loc=0., scale=1.).

Tensor of random values drawn from a uniform distribution between 0 and 1. Equivalent to np.random.uniform(size=(3, 1), low=0., high=1.).

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- A key difference between NumPy arrays and TensorFlow tensors:
  - ➢ TensorFlow tensors cannot change their values once created.
- In other words, a tensor in TensorFlow is a constant.
- Example: in NumPy, you could do the following…

Listing 3.3 NumPy arrays are assignable

```
import numpy as np
x = np.ones(shape=(2, 2))
x[0, 0] = 0.
```

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- Try to do the same thing in TensorFlow, and you will get an error: “EagerTensor object does not support item assignment.”

Listing 3.4 TensorFlow tensors are not assignable

```
x = tf.ones(shape=(2, 2))
x[0, 0] = 0.
```

This will fail, as a tensor isn’t assignable.

- But training a model requires updating its state (trainable tensors).
- If tensors cannot change, how can we update them?
- This is why TensorFlow provides variables.
- tf.Variable is a class that manages a mutable state in TensorFlow.

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- To create a variable, you need to provide some initial value, such as a random tensor.

Listing 3.5 Creating a TensorFlow variable

```
>>> v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
>>> print(v)
array([[-0.75133973],
       [-0.4872893 ],
       [ 1.6626885 ]], dtype=float32)>
```

- The state of a variable can be modified via its assign method, as follows.

Listing 3.6 Assigning a value to a TensorFlow variable

```
>>> v.assign(tf.ones((3, 1)))
array([[1.],
       [1.],
       [1.]], dtype=float32)>
```

3.5 First steps with TensorFlow

3.5.1 Constants and Variables

- It also works for a subset of the coefficients.

Listing 3.7 Assigning a value to a subset of a TensorFlow variable

```
>>> v[0, 0].assign(3.)
array([[3.],
       [1.],
       [1.]], dtype=float32)>
```

- Similarly, assign_add() and assign_sub() are efficient equivalents of += and -=, as shown next.

Listing 3.8 Using `assign_add()`

```
>>> v.assign_add(tf.ones((3, 1)))
array([[2.],
       [2.],
       [2.]], dtype=float32)>
```

3.5 First steps with TensorFlow

3.5.2 Tensor operations: Doing math in TensorFlow

- Just like NumPy, TensorFlow offers a large collection of tensor operations to expressmathematical formulas.

Listing 3.9 A few basic math operations

```
a = tf.ones((2, 2))
b = tf.square(a)
c = tf.sqrt(a)
d = b + c
e = tf.matmul(a, b)
e *= d
```

Take the square.

Take the square root.

Add two tensors (element-wise).

Take the product of two tensors (as discussed in chapter 2).

Multiply two tensors (element-wise).

3.5 First steps with TensorFlow

3.5.3 A second look at the GradientTape API

- So far, tensors in TensorFlow may look very similar to NumPy.
- But TensorFlow can do things NumPy cannot:
  - If an expression is differentiable, TensorFlow can compute its gradient.
- With GradientTape:

Listing 3.10 Using the `GradientTape`

```
input_var = tf.Variable(initial_value=3.)
with tf.GradientTape() as tape:
    result = tf.square(input_var)
gradient = tape.gradient(result, input_var)
```

3.5 First steps with TensorFlow

3.5.3 A second look at the GradientTape API

- So far, the inputs to tape.gradient() were always variables.
- In fact, you can use any tensor as input.
- By default, TensorFlow only tracks trainable variables.
- For constants, you must explicitly tell TensorFlow to watch them with tape.watch().

Listing 3.11 Using `GradientTape` with constant tensor inputs

```
input_const = tf.constant(3.)
with tf.GradientTape() as tape:
    tape.watch(input_const)
    result = tf.square(input_const)
gradient = tape.gradient(result, input_const)
```

3.6 Understanding core Keras APIs

▣ Basic Building Blocks of Deep Learning

- The basic data structure of a neural network is the layer.
- A layer takes one or more tensors as input and outputs one or more tensors.
- Most layers have a state, usually represented by weights.
- Weights are tensors learned by stochastic gradient descent.
- These weights contain the knowledge that the neural network learns.

3.6 Understanding core Keras APIs

▣ Core Elements of Deep Learning

- Each layer has its own input tensor shape and way of processing data.
- Examples:
  - Rank-2 tensor (samples, features) → processed with a densely connected layer (Dense in Keras).
  - This is also called a fully connected layer.
  - Rank-3 tensor (samples, timesteps, features) → usually processed with an RNN layer (e.g., LSTM) or 1D convolution layer (Conv1D).
  - Rank-4 tensor (image data) → typically processed with a 2D convolution layer (Conv2D).
- Layers can be thought of as LEGO blocks for deep learning.
- Thanks to Keras, deep learning models are built by connecting compatible layers into a useful transformation pipeline.

3.6 Understanding core Keras APIs

Layers: The Keras Layer Class

- A simple API needs a central abstraction to organize everything.
- In Keras, this abstraction is the Layer class.
- Everything in Keras is either a Layer or interacts closely with Layers.
- A Layer is an object that encapsulates:
  - State (weights)
  - Computation (forward pass)
- In practice:
  - Weights are usually defined in the build() method (though they can be in __init__()).
  - Computation is defined in the call() method.

3.6 Understanding core Keras APIs

Layers: Components of Deep Learning

- In the previous section, we defined a **NaiveDense class** with weights W and b
- The following example shows the **equivalent Keras layer**.

Listing 3.22 A `Dense` layer implemented as a `Layer` subclass

```
from tensorflow import keras

class SimpleDense(keras.layers.Layer):

    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = activation

    def build(self, input_shape):
        input_dim = input_shape[-1]
        self.W = self.add_weight(shape=(input_dim, self.units),
                                 initializer="random_normal")
        self.b = self.add_weight(shape=(self.units,),
                                 initializer="zeros")

    def call(self, inputs):
        y = tf.matmul(inputs, self.W) + self.b
        if self.activation is not None:
            y = self.activation(y)
        return y
```

All Keras layers inherit from the base Layer class.

Weight creation takes place in the build() method.

We define the forward pass computation in the call() method.

add_weight() is a shortcut method for creating weights. It is also possible to create standalone variables and assign them as layer attributes, like self.W = tf.Variable(tf.random.uniform(w_shape)).

=

```
import tensorflow as tf

class NaiveDense:
    def __init__(self, input_size, output_size, activation):
        self.activation = activation

        w_shape = (input_size, output_size)
        w_initial_value = tf.random.uniform(w_shape, minval=0, maxval=1e-1)
        self.W = tf.Variable(w_initial_value)

        b_shape = (output_size,
        b_initial_value = tf.zeros(b_shape)
        self.b = tf.Variable(b_initial_value)

    def __call__(self, inputs)::
        return self.activation(tf.matmul(inputs, self.W) + self.b)

    @property
    def weights(self):
        return [self.W, self.b]
```

Create a matrix, W, of shape (input_size, output_size), initialized with random values.

Create a vector, b, of shape (output_size,), initialized with zeros.

Apply the forward pass.

Convenience method for retrieving the layer's weights

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- Once you create an instance of this class, you can use it like a function that takes tensors as input.

```
>>> my_dense = SimpleDense(units=32, activation=tf.nn.relu)
>>> input_tensor = tf.ones(shape=(2, 784))
>>> output_tensor = my_dense(input_tensor)
>>> print(output_tensor.shape)
(2, 32))
```

Instantiate our layer, defined previously.

Create some test inputs.

Call the layer on the inputs, just like a function.

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- Like LEGO blocks, only compatible layers can be connected together.
- Layer compatibility:
  - ➢ Each layer only accepts inputs of certain shapes.
  - ➢ Each layer outputs tensors of specific shapes.

```
from tensorflow.keras import layers
layer = layers.Dense(32, activation="relu")
```

A dense layer with 32 output units

- This layer returns a tensor with 32 units in its first dimension.
- It can only connect to later layers that expect 32-dimensional input vectors.

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- With Keras, you usually don't need to worry about shape compatibility.
- Layers added to a model automatically adapt to the size of the previous layer.
- Example:

```
from tensorflow.keras import models
from tensorflow.keras import layers
model = models.Sequential([
    layers.Dense(32, activation="relu"),
    layers.Dense(32)
])
```

- These layers do not require explicit input size information.
- Instead, the input shape is inferred the first time the model sees input data.

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- In the simple NaiveDense class (from Chapter 2), each layer required the input size to be specified explicitly to create weights.
- This is not ideal.
- Example: a model built with NaiveDense layers requires knowing each layer's input size in advance:

```
model = NaiveSequential([
    NaiveDense(input_size=784, output_size=32, activation="relu"),
    NaiveDense(input_size=32, output_size=64, activation="relu"),
    NaiveDense(input_size=64, output_size=32, activation="relu"),
    NaiveDense(input_size=32, output_size=10, activation="softmax")
])
```

- This is inconvenient, since you must manually track the size of each previous layer.

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- The problem becomes worse when the rule for output size is complex.
- For instance, what if our layer returned outputs of shape (batch, input_size * 2 if input_size % 2 == 0 else input_size * 3)?
- If we were to reimplement our NaiveDense layer as a Keras layer capable of automatic shape inference, it would look like the previous SimpleDense layer (see listing 3.22), with its build() and call() methods.

3.6 Understanding core Keras APIs

Layers: Components of Deep Learning

- Unlike NaiveDense, the SimpleDense class does not create weights in the constructor.
- Instead, weights are created inside the dedicated build() method.

Listing 3.22 A Dense layer implemented as a Layer subclass

```
from tensorflow import keras

class SimpleDense(keras.layers.Layer):

    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = activation

    def build(self, input_shape):
        input_dim = input_shape[-1]
        self.W = self.add_weight(shape=(input_dim, self.units),
                                 initializer="random_normal")
        self.b = self.add_weight(shape=(self.units,),
                                 initializer="zeros")

    def call(self, inputs):
        y = tf.matmul(inputs, self.W) + self.b
        if self.activation is not None:
            y = self.activation(y)
        return y
```

All Keras layers inherit from the base Layer class.

Weight creation takes place in the build() method.

We define the forward pass computation in the call() method.

add_weight() is a shortcut method for creating weights. It is also possible to create standalone variables and assign them as layer attributes, like self.W = tf.Variable(tf.random.uniform(w_shape)).

```
import tensorflow as tf

class NaiveDense:
    def __init__(self, input_size, output_size, activation):
        self.activation = activation

        w_shape = (input_size, output_size)
        w_initial_value = tf.random.uniform(w_shape, minval=0, maxval=1e-1)
        self.W = tf.Variable(w_initial_value)

        b_shape = (output_size,)
        b_initial_value = tf.zeros(b_shape)
        self.b = tf.Variable(b_initial_value)

    def __call__(self, inputs):
        return self.activation(tf.matmul(inputs, self.W) + self.b)

    @property
    def weights(self):
        return [self.W, self.b]
```

Create a matrix, W, of shape (input_size, output_size), initialized with random values.

Create a vector, b, of shape (output_size,), initialized with zeros.

Apply the forward pass.

Convenience method for retrieving the layer's weights

3.6 Understanding core Keras APIs

▣ Layers: Components of Deep Learning

- This method receives the input shape as its argument when the layer is first called.
- The build() method is triggered automatically the first time the layer is called (via __call__()).
- This is why computation is defined in call(), while weight creation is in build().
- Example: the default __call__() method of a base Layer class:

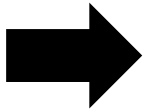
```
def __call__(self, inputs):
    if not self.built:
        self.build(inputs.shape)
        self.built = True
    return self.call(inputs)
```

3.6 Understanding core Keras APIs

Layers: Components of Deep Learning

- With automatic shape inference, the earlier example can be expressed simply:

```
model = NaiveSequential([
    NaiveDense(input_size=784, output_size=32, activation="relu"),
    NaiveDense(input_size=32, output_size=64, activation="relu"),
    NaiveDense(input_size=64, output_size=32, activation="relu"),
    NaiveDense(input_size=32, output_size=10, activation="softmax")
])
```



```
model = keras.Sequential([
    SimpleDense(32, activation="relu"),
    SimpleDense(64, activation="relu"),
    SimpleDense(32, activation="relu"),
    SimpleDense(10, activation="softmax")
])
```

- The Layer class's __call__() method does more than just shape inference:
- Manages switching between eager execution and graph execution.
- Handles tasks like input masking.
- For now: when implementing a custom layer, remember to put the forward pass logic inside the call() method.

3.6 Understanding core Keras APIs

▣ From Layers to Models

- A deep learning model is a graph of layers.
- In Keras, this corresponds to the Model class.
- So far, we've only seen the Sequential model (a simple stack of layers mapping one input to one output).
- Later, you will encounter many more complex architectures.
- Common structures include:
  - ➢ Two-branch networks
  - ➢ Multihead networks
  - ➢ Residual connections

3.6 Understanding core Keras APIs

From Layers to Models

- The network topology can become quite complex.
- Example:
  - Figure 3-9 shows the Transformer architecture,
  - A widely used structure for processing text data.

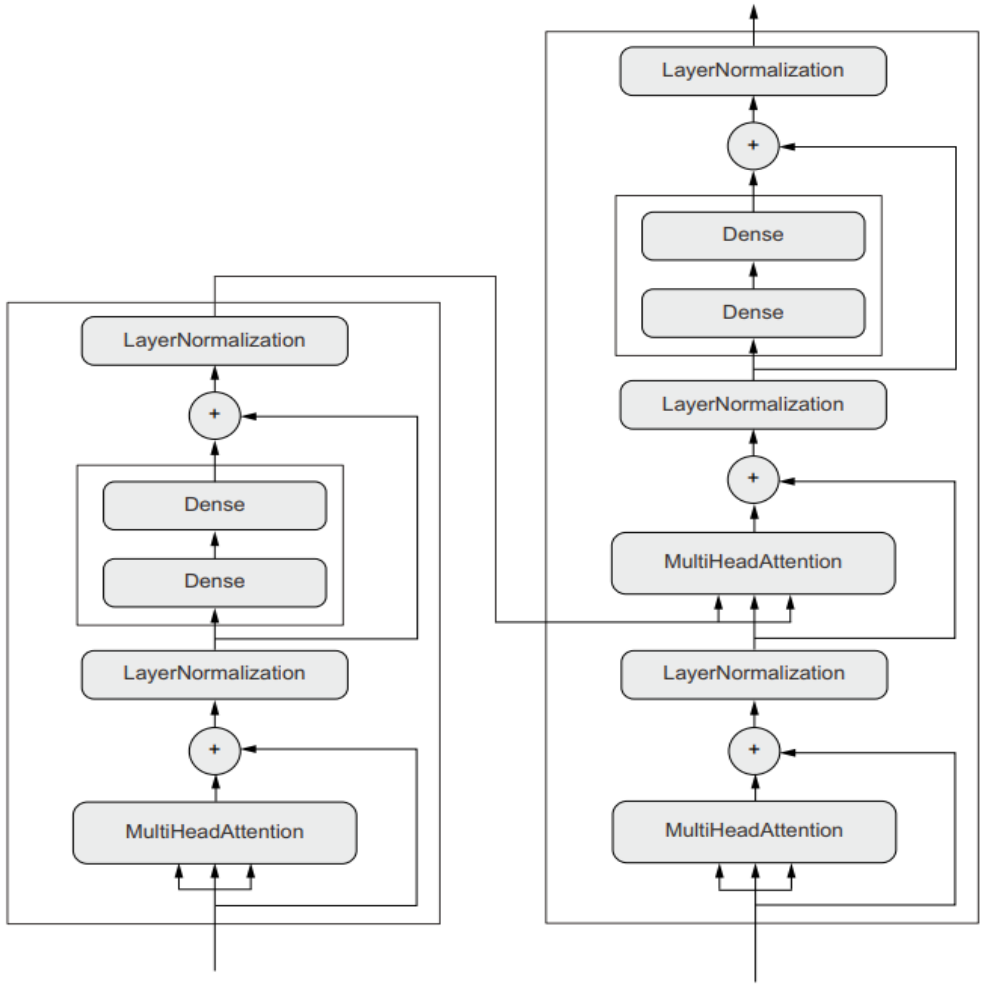


Figure 3.9 The Transformer architecture (covered in chapter 11). There's a lot going on here. Throughout the next few chapters, you'll climb your way up to understanding it.

3.6 Understanding core Keras APIs

▣ From Layers to Models

- In Keras, there are two common ways to build models:
  - ➢ Create a subclass of the Model class, or
  - ➢ Use the Functional API for more flexibility with less code.
- A model defines a hypothesis space.
- Choosing a network structure means choosing a space of possible functions that map input tensors to output tensors.
- The goal of training is to find the best set of weights for these tensor transformations.

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- After defining the model structure, you must choose three things:
- Loss function (objective function):
  - What the model minimizes during training.
  - A measure of success for the current task.
- Optimizer:
  - Decides how the network updates based on the loss.
  - Usually implemented with stochastic gradient descent (**SGD**) variants.
- Metrics:
  - Success measures to monitor during training and evaluation (e.g., accuracy).
  - Unlike loss, metrics are not directly optimized.
  - Metrics may not even be differentiable.

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- After selecting the loss, optimizer, and metrics, you can start training with:
  - ➢ The built-in compile() and fit() methods, or
  - ➢ A custom training loop.
- The compile() method sets up the training process.
- Its main arguments are:
  - ➢ optimizer
  - ➢ loss
  - ➢ metrics (a list)

```
model = keras.Sequential([keras.layers.Dense(1)])
model.compile(optimizer="rmsprop",
              loss="mean_squared_error",
              metrics=["accuracy"])
```

Define a linear classifier.

Specify the optimizer by name: RMSprop (it's case-insensitive).

Specify the loss by name: mean squared error.

Specify a list of metrics: in this case, only accuracy.

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- In compile(), you can specify the optimizer, loss, and metrics as strings (e.g., "rmsprop").
- These strings are just shortcuts that map to full Python objects.
- Example: "rmsprop" = keras.optimizers.RMSprop().
- You can also pass the actual class instances instead of strings.

```
model.compile(optimizer=keras.optimizers.RMSprop(),
              loss=keras.losses.MeanSquaredError(),
              metrics=[keras.metrics.BinaryAccuracy()])
```

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- Useful when you want to:
- Provide custom loss functions or metrics, or
- Configure objects in detail.
- Example: changing the learning_rate of an optimizer:

```
model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
              loss=my_custom_loss,
              metrics=[my_custom_metric_1, my_custom_metric_2])
```

- In practice, beginners usually don’t need to create their own losses, metrics, or optimizers.
- Keras already provides a wide range of built-in options.

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- Common optimizers in Keras:
  - ➢ SGD (with optional momentum)
  - ➢ RMSprop
  - ➢ Adam
  - ➢ Adagrad
  - ➢ Others

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- Common loss functions:
  - ➢ CategoricalCrossentropy
  - ➢ SparseCategoricalCrossentropy
  - ➢ BinaryCrossentropy
  - ➢ MeanSquaredError
  - ➢ KLDivergence
  - ➢ CosineSimilarity
  - ➢ Others

3.6 Understanding core Keras APIs

▣ Compile Step: Setting Up the Training Process

- Common evaluation metrics:
  - ➢ CategoricalAccuracy
  - ➢ SparseCategoricalAccuracy
  - ➢ BinaryAccuracy
  - ➢ AUC
  - ➢ Precision
  - ➢ Recall
  - ➢ Others

3.6 Understanding core Keras APIs

▣ Choosing the Right Loss Function

- Selecting the correct loss function is very important.
- The network minimizes the loss, so the choice directly shapes model behavior.
- If the loss is unrelated to the actual task, the model may do harmful things.
- Example thought experiment:
  - Suppose an AI is trained to maximize average happiness.
  - It may decide to kill most people and keep only a few happy ones → average happiness increases.
- This shows why the wrong loss function leads to bad outcomes.
- Neural networks only try to minimize the chosen loss.
- If you choose poorly, the model will produce undesired side effects.

3.6 Understanding core Keras APIs

▣ Choosing the Right Loss Function

- Fortunately, for common tasks like classification, regression, and sequence prediction, there are simple guidelines.
- Examples:
  - ➢ For binary classification → use BinaryCrossentropy.
  - ➢ For multi-class classification → use CategoricalCrossentropy.
- You will only need to design your own loss function if doing entirely new research.

3.6 Understanding core Keras APIs

▣ Understanding the fit() Method

- After compile(), call the fit() method.
- The fit() method runs the training loop.
- Key arguments of fit():
  - ➢ Training data (inputs and targets): usually NumPy arrays, tensors, or Dataset objects.
  - ➢ Epochs: number of times the training loop goes over the entire dataset.
  - ➢ Batch size: number of samples per batch for stochastic gradient descent (SGD).
    - Defines how many samples are used to compute each gradient update.

3.6 Understanding core Keras APIs

▣ Understanding the fit() method

- After compile() comes fit(). The fit() method implements the training loop itself.

Listing 3.23 Calling `fit()` with NumPy data

```
history = model.fit(
    inputs,
    targets,
    epochs=5,
    batch_size=128
)
```

The input examples, as a NumPy array

The corresponding training targets, as a NumPy array

The training loop will iterate over the data 5 times.

The training loop will iterate over the data in batches of 128 examples.

3.6 Understanding core Keras APIs

▣ Understanding the fit() method

- Calling fit() returns a History object.
- This object has a history attribute (a dictionary).
- The dictionary maps:
  - ➢ Keys: "loss" or metric names
  - ➢ Values: lists of results for each epoch

```
>>> history.history
{"binary_accuracy": [0.855, 0.9565, 0.9555, 0.95, 0.951],
 "loss": [0.6573270302042366,
          0.07434618508815766,
          0.07687718723714351,
          0.07412414988875389,
          0.07617757616937161]}
```

3.6 Understanding core Keras APIs

▣ Monitoring Loss and Metrics on Validation Data

- The goal of machine learning is not just to perform well on the training data.
- That would be easy: simply follow the gradients.
- **The true goal is to perform well in general, especially on data never seen before.**
- Performing well on training data does not guarantee performance on new data.
- Example:
  - ➢ A model could simply memorize the relationship between samples and targets.
  - ➢ Such a model would fail to predict targets for unseen data.

3.6 Understanding core Keras APIs

▣ Monitoring Loss and Metrics on Validation Data

- To estimate performance on new data, set aside part of the training data as validation data.
- The model is not trained on validation data, but its loss and metrics are computed on it.
- To do this, use the validation_data argument in the fit() method.
- Like training data, validation data can be provided as NumPy arrays, tensors, or Dataset objects.

3.6 Understanding core Keras APIs

▣ Monitoring Loss and Metrics on Validation Data

Listing 3.24 Using the `validation_data` argument

```
model = keras.Sequential([keras.layers.Dense(1)])
model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=0.1),
              loss=keras.losses.MeanSquaredError(),
              metrics=[keras.metrics.BinaryAccuracy()])

indices_permutation = np.random.permutation(len(inputs))
shuffled_inputs = inputs[indices_permutation]
shuffled_targets = targets[indices_permutation]

num_validation_samples = int(0.3 * len(inputs))
val_inputs = shuffled_inputs[:num_validation_samples]
val_targets = shuffled_targets[:num_validation_samples]
training_inputs = shuffled_inputs[num_validation_samples:]
training_targets = shuffled_targets[num_validation_samples:]
model.fit(
    training_inputs,
    training_targets,
    epochs=5,
    batch_size=16,
    validation_data=(val_inputs, val_targets)
)
```

To avoid having samples from only one class in the validation data, shuffle the inputs and targets using a random indices permutation.

Reserve 30% of the training inputs and targets for validation (we'll exclude these samples from training and reserve them to compute the validation loss and metrics).

Training data, used to update the weights of the model

Validation data, used only to monitor the validation loss and metrics

3.6 Understanding core Keras APIs

▣ Monitoring Loss and Metrics on Validation Data

- The loss on validation data is called validation loss, to distinguish it from training loss.
- It's essential to keep training and validation data strictly separate.
- Purpose: to monitor whether the model is truly useful on new data.
- If validation data leaks into training, validation loss and metrics become misleading.
- After training, you can compute validation loss and metrics with the evaluate() method.

```
loss_and_metrics = model.evaluate(val_inputs, val_targets, batch_size=128)
```

3.6 Understanding core Keras APIs

▣ Monitoring Loss and Metrics on Validation Data

- The evaluate() method iterates over the given data in batches (batch_size) and returns a list of scalar values.
- In the returned list:
  - ➢ The first item = validation loss.
  - ➢ The following items = metric values for validation data.
- If no metrics were specified during model setup, only the validation loss is returned.

3.6 Understanding core Keras APIs

▣ Inference: Using a Trained Model

- After training, you can use the model to make predictions on new data.
- This process is called inference.
- The simplest way is to call the model's __call__() method:

```
predictions = model(new_inputs)
```

Takes a NumPy array or TensorFlow tensor and returns a TensorFlow tensor

- This processes all inputs in new_inputs at once.
- If the dataset is very large, this may be impractical (especially if GPU memory is limited).

3.6 Understanding core Keras APIs

▣ Inference: Using a Trained Model

- A better way to do inference is using the predict() method.
- This method:
  - ➢ Processes the data in small batches.
  - ➢ Returns predictions as NumPy arrays.
- Unlike __call__(), it can also handle Dataset objects.

3.6 Understanding core Keras APIs

▣ Inference: Using a Trained Model

- Example: calling the trained model's predict() method with validation data.
- Returns a scalar score for each input sample, representing the model's prediction.

```
>>> predictions = model.predict(val_inputs, batch_size=128)
>>> print(predictions[:10])
[[0.3590725 ]
 [0.82706255]
 [0.74428225]
 [0.682058  ]
 [0.7312616 ]
 [0.6059811 ]
 [0.78046083]
 [0.025846  ]
 [0.16594526]
 [0.72068727]]
```

3.7 Summary

▣ Summary

- TensorFlow is a powerful computing framework that runs on CPU, GPU, and TPU.
  - ➢ Can automatically compute gradients for any differentiable expression.
  - ➢ Programs can run on many devices and be converted to work with different runtimes (e.g., JavaScript).
- Keras is the standard API for deep learning with TensorFlow, and the main library used in this book.
- Core TensorFlow objects:
  - ➢ Tensors, Variables, Tensor operations, GradientTape.
- Core Keras class:
  - ➢ Layer → encapsulates weights and computation.
  - ➢ Layers are combined to build models.

3.7 Summary

▣ Summary

- Before training, specify the optimizer, loss, and metrics with the compile() method.
- Train the model using mini-batch stochastic gradient descent with the fit() method.
- During training, fit() can also monitor validation loss and metrics on unseen data.
- After training, use the predict() method to generate predictions on new inputs.



Q & A

Prof. Chang-Hee Han